

## **TERRAFORM**

Terraform is an open-source Infrastructure as Code (IaC) tool used to automate the creation, update, and deletion of cloud resources such as EC2, VPC, S3 and provisioning of infrastructure resources using configuration files.

### **Access Key and Secret Access Key**

Terraform uses AWS Access Key and Secret Access Key for authentication and authorization to access AWS services.

### **HCL (HashiCorp Configuration Language)**

HCL is the configuration language used in Terraform to define cloud resources and infrastructure in a simple and human-readable format.

### **State File**

- A Terraform state file (terraform.tfstate) is used to store information about managed infrastructure resources and track the current infrastructure state.
- It helps Terraform track what resources are created, updated, or deleted.

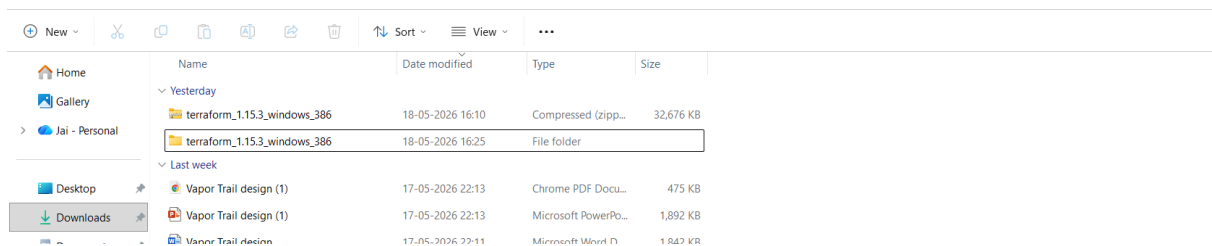
### **Terraform Workflow**

- **terraform init** - Initializes Terraform working directory and downloads required provider plugins.
- **terraform plan** - Displays the execution plan and shows infrastructure changes before applying them.
- **terraform apply** - Creates or updates infrastructure resources based on configuration files.
- **terraform destroy** - Deletes all infrastructure resources managed by Terraform.

# HANDS-ON FOR CREATING AND DELETING INSTANCE USING TERRAFORM

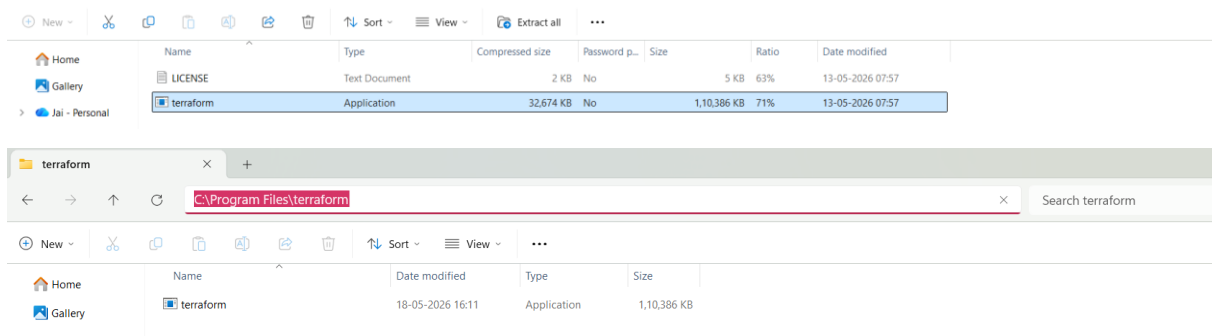
## Screenshot: Download and Extract Terraform

- Open browser and download Terraform from official website
- Download Terraform ZIP file
- Extract the downloaded ZIP file



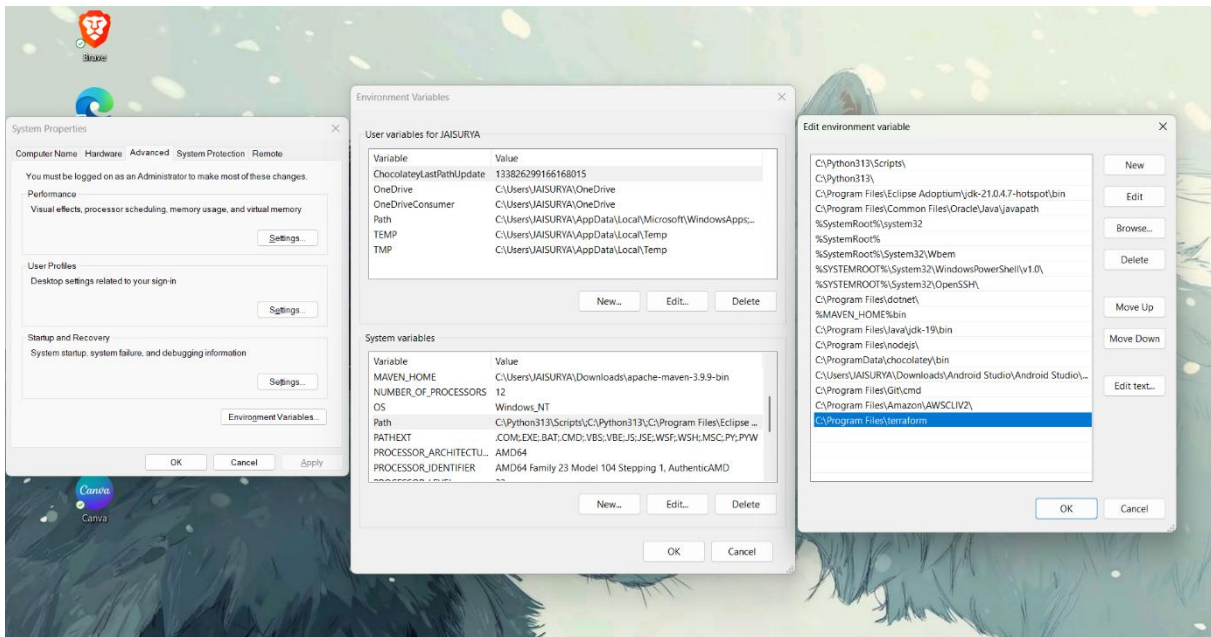
## Screenshot: Configure Terraform Folder

- Open extracted Terraform folder
- Copy Terraform application file (terraform.exe)
- Open This PC → Program Files
- Create a new folder named Terraform
- Paste terraform.exe file into the Terraform folder
- Copy the Terraform folder path from File Explorer address bar



## Screenshot: Add Terraform Path to Environment Variables

- Open Edit the system environment variables
- Click Environment Variables
- In System Variables, select Path
- Click Edit → New
- Paste the Terraform folder path
- Click OK to save changes



## Screenshot: Verify AWS CLI and Terraform Installation

- Open Command Prompt (cmd)
- Execute `aws --version` (to verify AWS CLI installation)
- Execute `terraform --version` (to verify Terraform installation)

```

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\JAISURYA>aws --version
aws-cli/2.34.2 Python/3.13.11 Windows/11 exe/AMD64

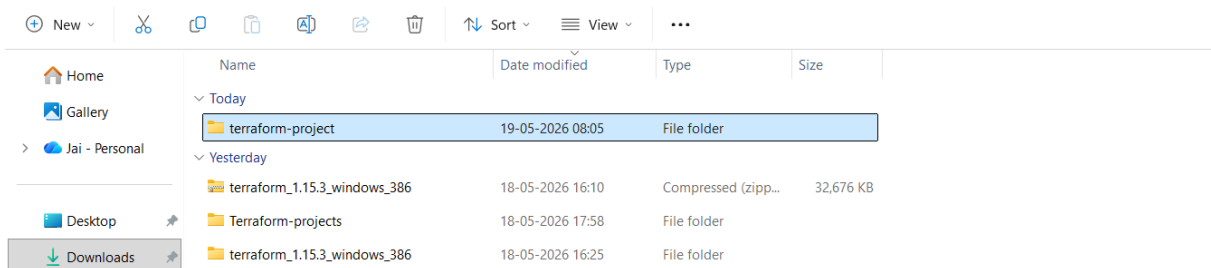
C:\Users\JAISURYA>terraform --version
Terraform v1.15.3
on windows_386

C:\Users\JAISURYA>

```

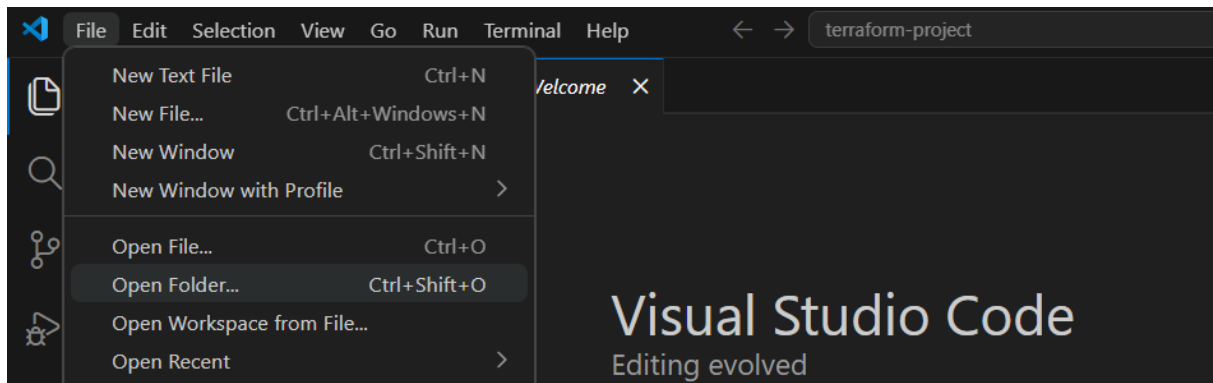
## Screenshot: Create Local Project Folder

- Open Downloads folder
- Create a new empty folder for Terraform project files



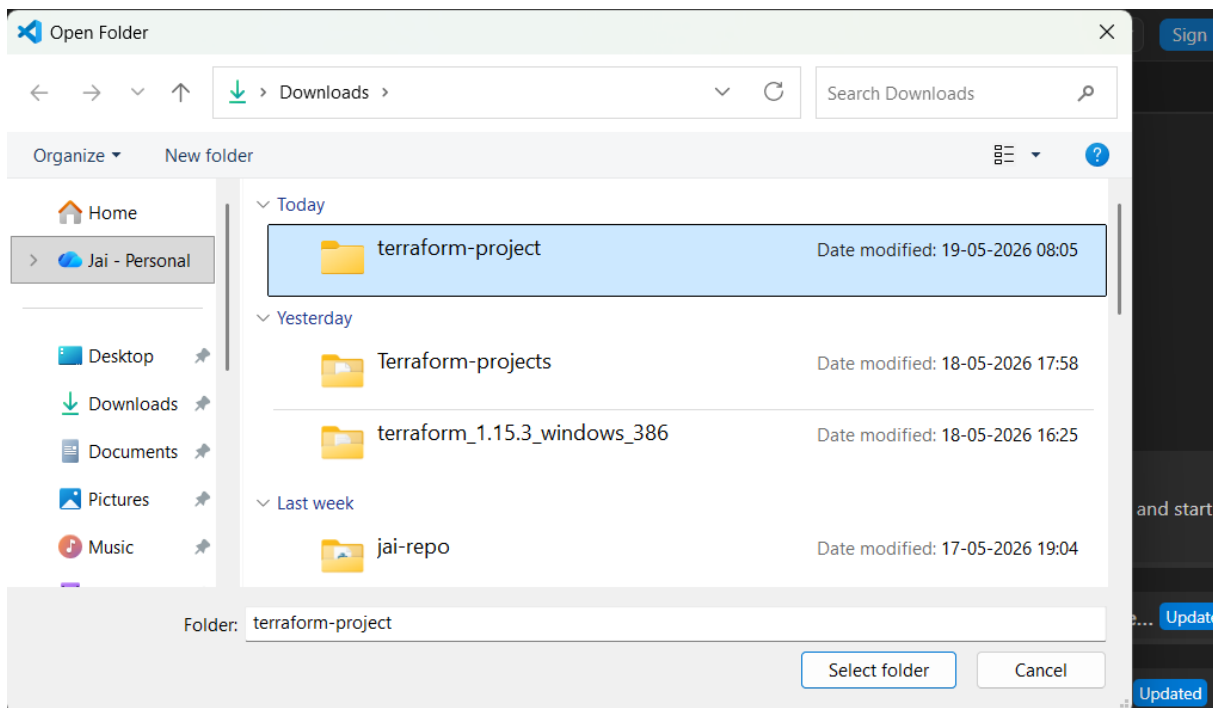
### Screenshot: Open Project Folder in VS Code

- Open Visual Studio Code
- Click Open Folder



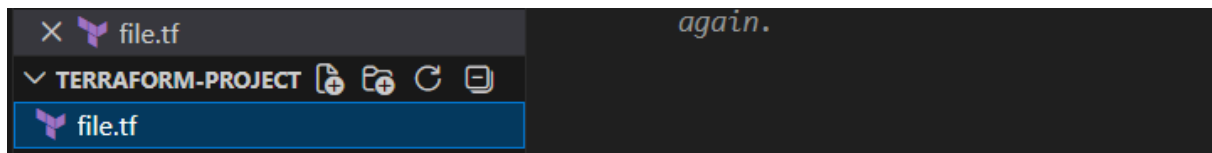
### Screenshot: Select Project Folder in VS Code

- Select the created Terraform project folder
- Click Select Folder
- Folder opened successfully in VS Code



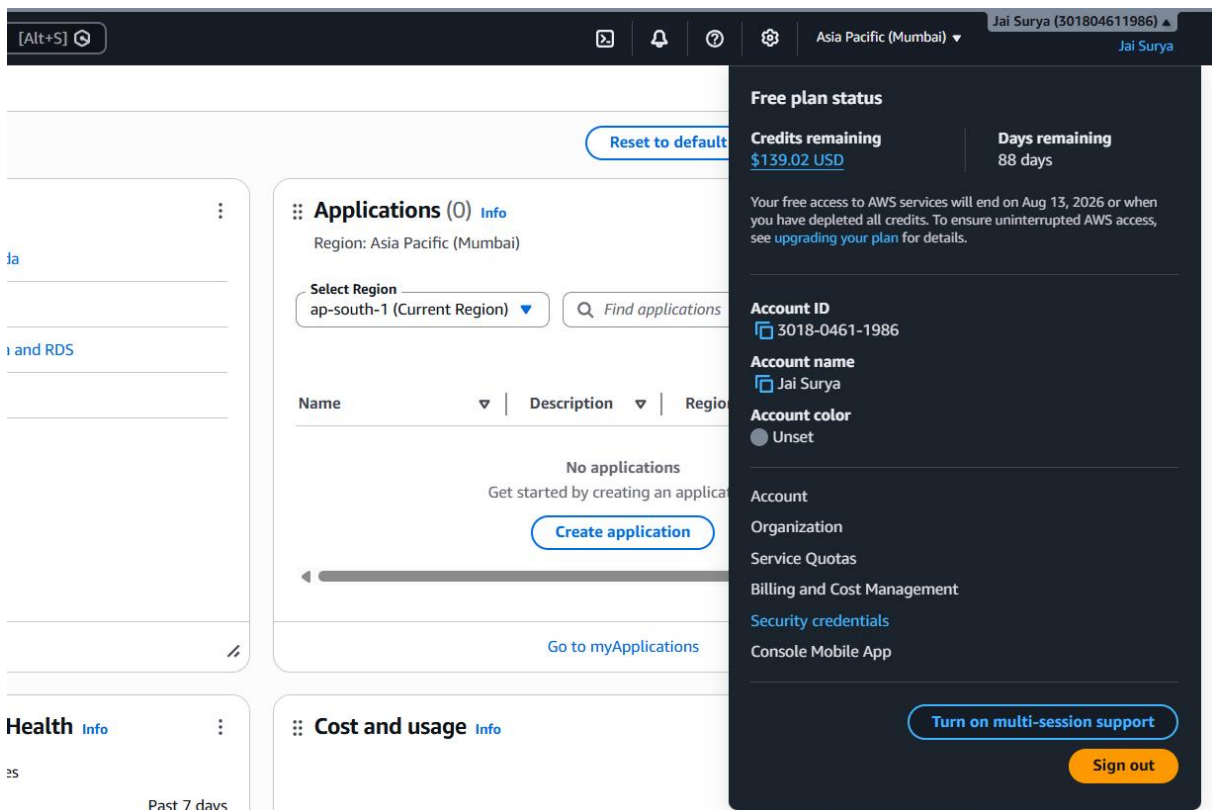
### Screenshot: Create Terraform File

- Create a new file in VS Code
- Enter file name with .tf extension (example: main.tf)



## Screenshot: Open AWS Security Credentials

- Open AWS Management Console
- Click Profile name
- Click Security credentials
- Open access and security settings page



## Screenshot: Generate AWS Access Key and Secret Access Key

- Click Create access key
- Tick the confirmation check box
- Click Create access key
- AWS Access Key and Secret Access Key generated successfully

**Access keys (0)**

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time.

| Access key ID  | Created on | Access key last used | Region last used | Service last used | Status |
|----------------|------------|----------------------|------------------|-------------------|--------|
| No access keys |            |                      |                  |                   |        |

As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. [Learn more](#)

[Create access key](#)

---

**Step 1: Alternatives to root user access keys**

**Alternatives to root user access keys**

**Root user access keys are not recommended**

Root user access keys have long-term unlimited permissions that can't be restricted. Instead, when accessing AWS CLI, SDKs, or tools for local development with AWS, use the `aws login` command and your existing console credentials for access.

[Learn more about alternatives to root user access keys](#)

**Continue to create access key?**

☒ I understand creating a root access key is not a best practice, but I still want to create one.

[Cancel](#) [Create access key](#)

---

**Step 2: Retrieve access key**

**Retrieve access key**

**Access key**

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

| Access key            | Secret access key          |
|-----------------------|----------------------------|
| AKIAUMRHUSGJJJBH2BY6Q | ***** <a href="#">Show</a> |

## Screenshot: Note AMI ID and Region

- Go to EC2
- Click Launch Instance
- Note the AMI ID of the selected instance
- Note the AWS Region name used for the instance launch

**Summary**

**Number of instances** [Info](#)

1

**Software Image (AMI)**

Amazon Linux 2023 AMI 2023.11....[read more](#)

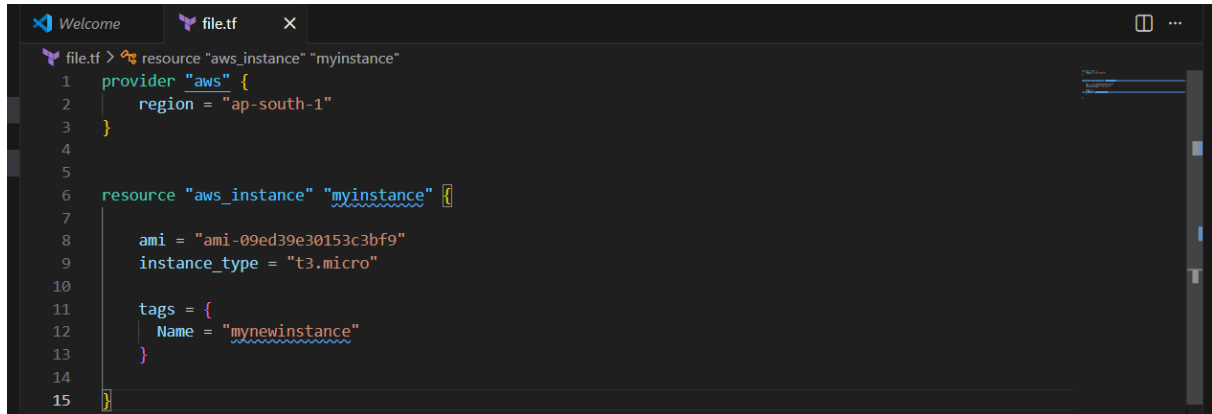
[ami-09ed39e30153c3bf9](#)

**Virtual server type (instance type)**

t3.micro

### Screenshot: Add Terraform Configuration Code

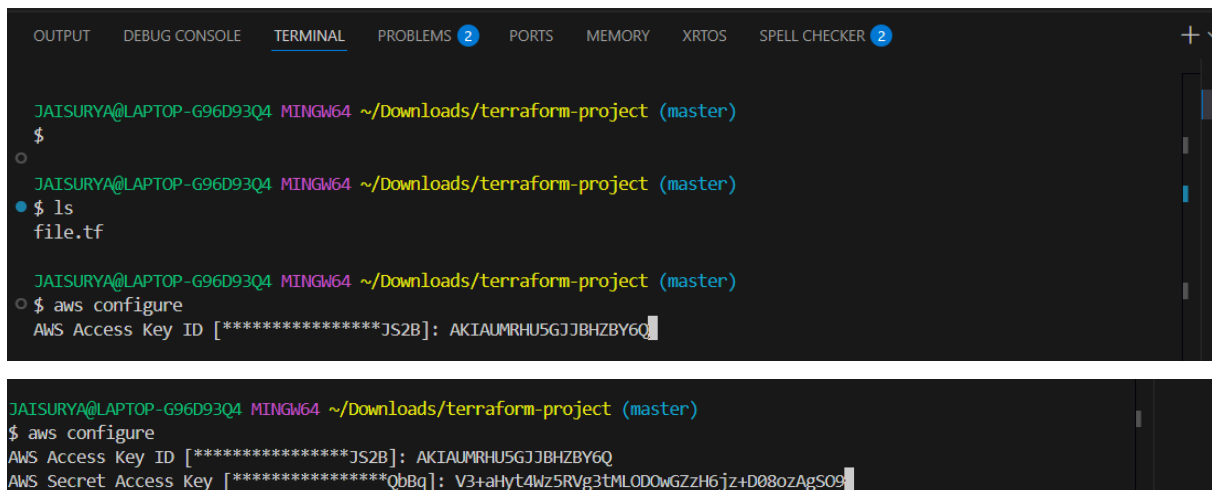
- Paste the Terraform code into the .tf file
- Configure AWS provider, region, access key, secret access key, and EC2 instance details
- Terraform configuration file saved successfully



```
file.tf > resource "aws_instance" "myinstance"
1  provider "aws" {
2      region = "ap-south-1"
3  }
4
5
6  resource "aws_instance" "myinstance" {
7
8      ami = "ami-09ed39e30153c3bf9"
9      instance_type = "t3.micro"
10
11      tags = {
12          Name = "mynewinstance"
13      }
14
15  }
```

### Screenshot: Configure AWS CLI in VS Code Terminal

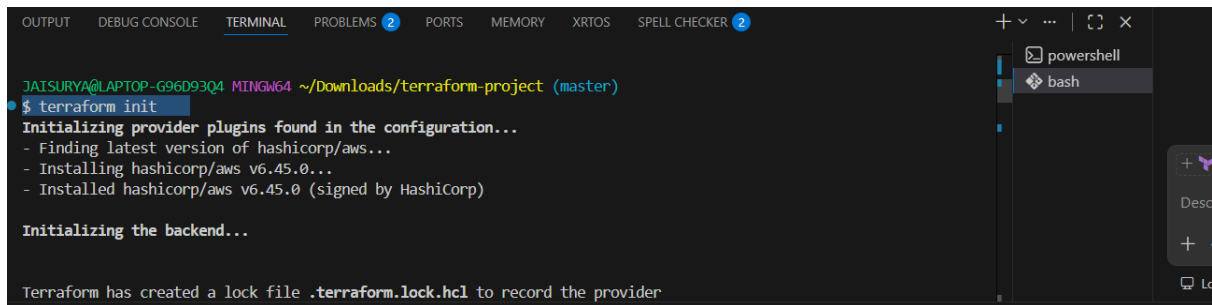
- Open Terminal in VS Code
- Select Git Bash terminal
- Execute aws configure
- Paste AWS Access Key ID
- Paste AWS Secret Access Key
- Enter default region name
- Enter output format if required



```
OUTPUT  DEBUG CONSOLE  TERMINAL  PROBLEMS 2  PORTS  MEMORY  XRTOS  SPELL CHECKER 2
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$ ls
file.tf
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$ aws configure
AWS Access Key ID [*****JS2B]: AKIAUMRHU5GJJJBHZBY6Q
AWS Secret Access Key [*****QbBq]: V3+aHyt4Wz5RVg3tML0D0wGZzH6jz+D08ozAgS09
```

### Screenshot: Initialize Terraform Working Directory

- Execute terraform init
- Terraform initializes the working directory
- Required AWS provider plugins downloaded successfully
- Terraform initialization completed successfully



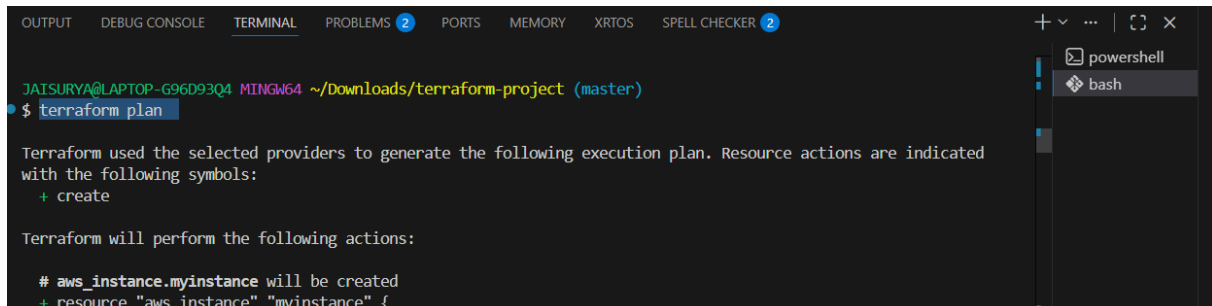
```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.45.0...
- Installed hashicorp/aws v6.45.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
```

### Screenshot: Terraform Plan Execution

- Execute terraform plan
- Terraform displays execution plan
- Shows resources that will be created, updated, or deleted
- EC2 instance creation details displayed successfully



```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
+ create

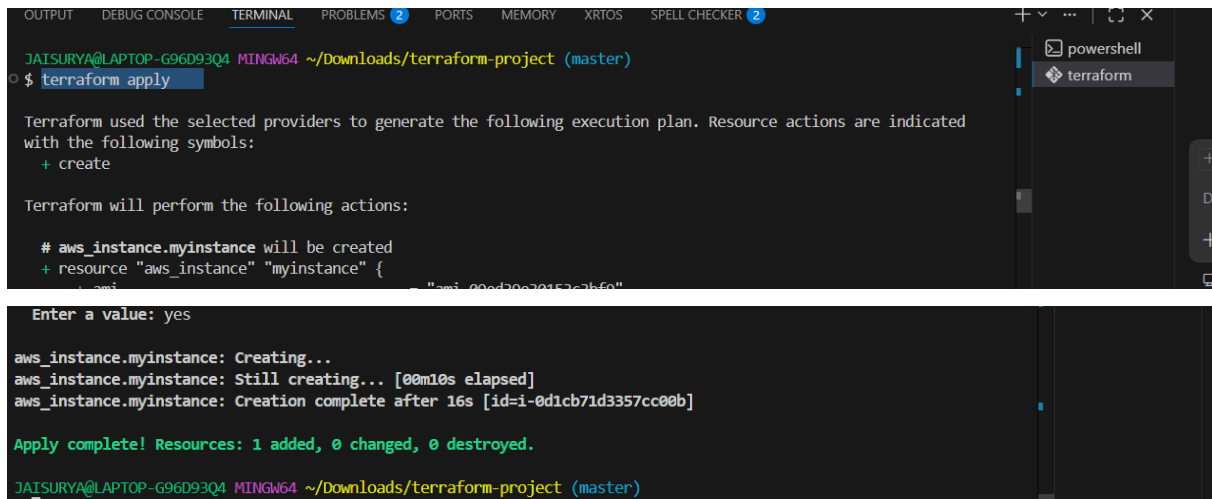
Terraform will perform the following actions:

# aws_instance.myinstance will be created
+ resource "aws_instance" "myinstance" {
```

### Screenshot: Apply Terraform Configuration

- Execute terraform apply
- Terraform displays resource creation confirmation
- Type yes to approve infrastructure creation
- EC2 instance created successfully in AWS using Terraform





OUTPUT DEBUG CONSOLE TERMINAL PROBLEMS 2 PORTS MEMORY XRTOS SPELL CHECKER 2

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.myinstance will be created
+ resource "aws_instance" "myinstance" {
  + ami           = "ami-00ed70e20152e3bfe"
  + availability_...
  + instance_type = "t3.micro"
  + key_name       = ""
  + subnet_id      = "subnet-00ed70e20152e3bfe"
  + vpc_id         = "vpc-00ed70e20152e3bfe"
}

Enter a value: yes

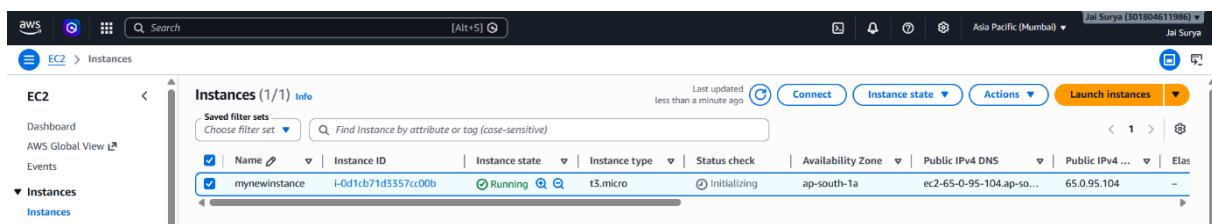
aws_instance.myinstance: Creating...
aws_instance.myinstance: Still creating... [00m10s elapsed]
aws_instance.myinstance: Creation complete after 16s [id=i-0d1cb71d3357cc00b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
```

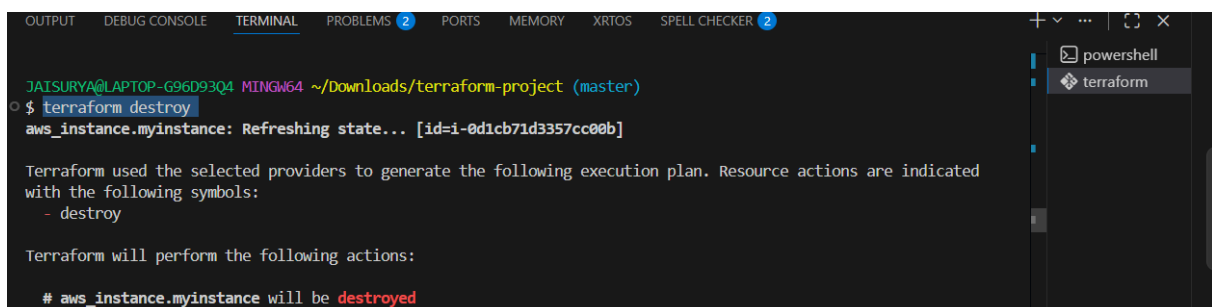
## Screenshot: Verify EC2 Instance Creation

- Open AWS Management Console
- Go to EC2 Dashboard
- Open Instances section
- EC2 instance created successfully using Terraform



## Screenshot: Destroy Terraform Infrastructure

- Execute terraform destroy
- Terraform displays resources that will be deleted
- Type yes to approve infrastructure deletion
- EC2 instance deleted successfully from AWS using Terraform



OUTPUT DEBUG CONSOLE TERMINAL PROBLEMS 2 PORTS MEMORY XRTOS SPELL CHECKER 2

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$ terraform destroy

aws_instance.myinstance: Refreshing state... [id=i-0d1cb71d3357cc00b]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
- destroy

Terraform will perform the following actions:

# aws_instance.myinstance will be destroyed
```

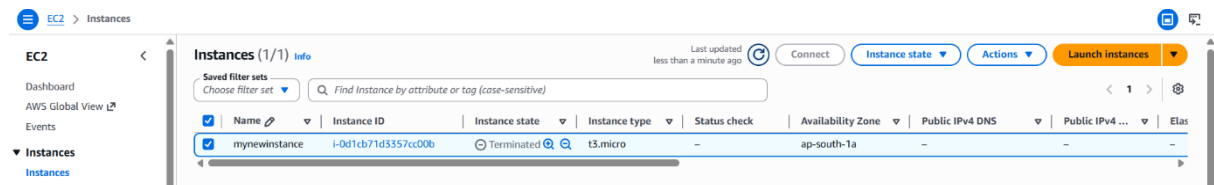
```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/terraform-project (master)
$ terraform destroy
Enter a value: yes

aws_instance.myinstance: Destroying... [id=i-0d1cb71d3357cc00b]
aws_instance.myinstance: Still destroying... [id=i-0d1cb71d3357cc00b, 00m10s elapsed]
aws_instance.myinstance: Still destroying... [id=i-0d1cb71d3357cc00b, 00m20s elapsed]
aws_instance.myinstance: Still destroying... [id=i-0d1cb71d3357cc00b, 00m30s elapsed]
aws_instance.myinstance: Destruction complete after 30s

Destroy complete! Resources: 1 destroyed.
```

## Screenshot: Verify EC2 Instance Termination

- Open AWS Management Console
- Go to EC2 Dashboard
- Open Instances section
- EC2 instance terminated successfully using Terraform



The screenshot shows the AWS Management Console's EC2 Instances page. The instance 'mynewinstance' is listed with the ID 'i-0d1cb71d3357cc00b' and is in a 'Terminated' state. The table below summarizes the instance details.

| Name          | Instance ID         | Instance state | Instance type | Status check | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Elas |
|---------------|---------------------|----------------|---------------|--------------|-------------------|-----------------|-----------------|------|
| mynewinstance | i-0d1cb71d3357cc00b | Terminated     | t3.micro      | -            | ap-south-1a       | -               | -               | -    |